

For the record, my system has an AMD triple-core, and runs Fedora 13 with the 2.6.34 vanilla kernel. If you don't have one, you can follow these steps to install a fresh vanilla kernel on your system:

1. Download the kernel source from www.kernel.org.
2. Extract the tar ball with `tar -jxvf linux-2.6.34.tar.bz2`.
3. Go to the extracted directory and run the following commands:

```
# make menuconfig
# make
# make modules_install
# make install
```

4. Finally, reboot your system, and choose the newly compiled kernel in the boot-loader screen.

About Kprobe

Kprobe is a very simple method to probe the running kernel. At a fundamental level, it requires the address of a kernel function that needs to be debugged. Then, you create pre- and post-handlers that will print a debugging message when the target kernel function is called. (Actually, a handler performs any action specified in its code; in this case, it happens to be printing.) Thus, every time that function is called, you can track it.

An example

To keep things simple, I have created a small and easy-to-understand example. The target kernel function is `'ip_rcv()'`. The Kprobe example kernel module is as follows:

```
#include<linux/module.h>
#include<linux/version.h>
#include<linux/kernel.h>
#include<linux/init.h>
#include<linux/kprobes.h>

static unsigned int counter = 0;
int Pre_Handler(struct kprobe *p, struct pt_regs *regs){
    printk("Pre_Handler: counter=%u\n",counter++);
    return 0;
}

void Post_Handler(struct kprobe *p, struct pt_regs *regs,
unsigned long flags){
    printk("Post_Handler: counter=%u\n",counter++);
}

static struct kprobe kp;

int myinit(void)
{
    printk("module inserted\n");
```

```
    kp.pre_handler = Pre_Handler;
    kp.post_handler = Post_Handler;
    kp.addr = (kprobe_opcode_t *)0xc071c9a9;
    register_kprobe(&kp);
    return 0;
}

void myexit(void)
{
    unregister_kprobe(&kp);
    printk("module removed\n");
}

module_init(myinit);
module_exit(myexit);
MODULE_AUTHOR("Manoj");
MODULE_DESCRIPTION("KPROBE MODULE");
MODULE_LICENSE("GPL");
```

The makefile required to build the kernel module object file that you need to insert into the kernel is as follows:

```
obj-m +=mod1.o mod2.o
KDIR= /lib/modules/${shell uname -r}/build
all:
    $(MAKE) -C $(KDIR) SUBDIRS=$(PWD) modules
clean:
```

```
rm -rf *.o *.ko *.mod.* .c* .t*
```

Code walk-through

Here's an explanation for the less obvious sections of the code.

```
struct kprobe kp;
```

To make use of *kprobe* functionality, you must declare a variable of the structure *struct kprobe*, which is declared in *include/linux/kprobes.h*. Here's a little extract:

```
struct kprobe {
    .
    .
    kprobe_opcode_t *addr;
    kprobe_pre_handler_t pre_handler;
    kprobe_post_handler_t post_handler;
}
```

The three members listed above are of interest to us. You need to assign the kernel address of the target function to the *addr* member; you can retrieve the address from the */proc/kallsyms* file, as follows:

```
# cat /proc/kallsyms | grep ip_rcv
c071c3e0 t ip_rcv_finish
c071c9a9 T ip_rcv
```